

UNIT-IV: Project Management: Risk Management, Managing People, Teamwork

Chapter 22 – Project Management

1. Software Project Management Software project management is an essential discipline concerned with ensuring that software is delivered to customers on time, within schedule, and in accordance with the requirements set by both the developing and procuring organizations. It is crucial because software development inherently operates under budget and schedule constraints imposed by the organization undertaking the development.

2. Success Criteria for Software Projects A software project is typically deemed successful if it meets the following criteria:

- **Timely Delivery:** The software is delivered to the customer at the agreed-upon time.
- **Budget Adherence:** The overall costs are kept within the allocated budget.
- **Customer Expectations Met:** The delivered software fulfills the customer's expectations.
- **Team Cohesion:** A coherent and well-functioning development team is maintained throughout the project lifecycle.

3. Software Management Distinctions Software project management differs from other forms of engineering management due to several unique characteristics of software:

- **Intangibility of the Product:** Software is an intangible product; it cannot be physically seen or touched. This makes it difficult for software project managers to gauge progress merely by observing the constructed artifact.
- **'One-Off' Projects:** Many large software projects are unique or 'one-off' endeavors, differing significantly from previous projects. This distinctiveness means that even experienced managers may find it challenging to anticipate all potential problems.
- **Variable and Organization-Specific Processes:** Software processes are often variable and tailored to specific organizational contexts. This variability means that predicting when a particular software process might lead to development problems is still not reliably possible.

4. Factors Influencing Project Management Project managers operate in diverse ways across different organizations due to several influencing factors:

- Company size
- Software customers
- Software size
- Software type
- Organizational culture
- Software development processes

5. Universal Management Activities Despite the influencing factors that lead to varied management approaches, certain activities are universally undertaken by project managers:

- **Project Planning:** Project managers are responsible for planning, estimating, scheduling project development, and assigning tasks to personnel. (Further details on this are typically covered in Chapter 23 of the source material).
- **Risk Management:** Project managers identify and assess potential risks that could affect a project, continuously monitor these risks, and implement actions when problems arise.
- **People Management:** This involves selecting suitable individuals for the project team and establishing effective working practices that foster high team performance.
- **Reporting:** Project managers are usually tasked with reporting project progress to both customers and the management of the developing company.
- **Proposal Writing:** The initial phase of a software project often includes writing a proposal to secure a contract. This proposal outlines the project's objectives and the methodology for its execution.

Risk Management

1. Definition and Importance of Risk Management Risk management involves identifying potential risks and formulating plans to minimize their negative effects on a project. It is particularly crucial in software development due to the inherent uncertainties within the field. These uncertainties stem from various sources, including loosely defined requirements, changes in requirements due to evolving customer needs, difficulties in accurately estimating the required time and resources, and variations in individual skills among team members. Effective risk management necessitates anticipating risks, understanding their potential impact on the project, product, and business, and taking proactive steps to avoid or mitigate them.

2. Risk Classification Risks are primarily classified along two dimensions:

- **Type of Risk:** This refers to the nature of the risk, such as technical, organizational, or other categories.
- **Affected Area:** This dimension specifies what aspects of the project are impacted by the risk:
 - **Project Risks:** These risks directly affect the project's schedule or resources.
 - **Product Risks:** These risks impact the quality or performance of the software being developed.
 - **Business Risks:** These risks affect the organization responsible for developing or procuring the software.

Examples of Project, Product, and Business Risks:

Risk	Affects	Description
Staff turnover	Project	Experienced staff leaving the project before its completion.
Management change	Project	A change in organizational management leading to different priorities.
Hardware unavailability	Project	Essential hardware for the project not being delivered on schedule.

Requirements change	Project and Product	A larger number of changes to requirements than anticipated.
Specification delays	Project and Product	Specifications of essential interfaces not being available on schedule.
Size underestimate	Project and Product	The system's size being underestimated.
CASE tool underperformance	Product	Computer-Aided Software Engineering (CASE) tools not performing as expected.
Technology change	Business	The underlying technology on which the system is built becoming superseded by newer technology.

3. The Risk Management Process Risk management is a continuous process comprising four main stages:

- **Risk Identification:** The initial step involves identifying potential project, product, and business risks.
- **Risk Analysis:** This stage assesses the likelihood of each identified risk occurring and evaluates the consequences if it does.
- **Risk Planning:** Plans are developed to either avoid the risks or minimize their potential effects.
- **Risk Monitoring:** Risks are continuously monitored throughout the project lifecycle to track their status and assess any changes.

4. Risk Identification Risk identification can be a collaborative team activity or based on the individual project manager's experience. A common method involves using a checklist of recurring risks, categorized by type:

- Technology risks
- Organizational risks
- People risks
- Requirements risks
- Estimation risks

Examples of Different Risk Types:

Risk Type	Possible Risks
Estimation	The time required to develop the software is underestimated. The rate of defect repair is underestimated. The size of the software is underestimated.
Organizational	The organization is restructured, leading to different management being responsible for the project. Organizational financial problems force reductions in the project budget.
People	It is impossible to recruit staff with the required skills. Key staff are ill and unavailable at critical times. Required training for staff is not available.
Requirements	Changes to requirements are proposed that necessitate major design rework. Customers fail to understand the impact of requirements changes.
Technology	The database used in the system cannot process as many transactions per second as expected. Reusable software components contain defects,

	rendering them unsuitable for planned reuse. Software tools cannot be integrated. Code generated by code generation tools is inefficient.
--	---

5. Risk Analysis Risk analysis involves evaluating both the probability and seriousness of each identified risk.

- **Probability:** The likelihood of a risk occurring can be categorized as very low, low, moderate, high, or very high.
- **Consequences:** The impact of a risk can be classified as catastrophic, serious, tolerable, or insignificant.

Risk Types, Probabilities, and Effects:

Risk	Probability	Effects
Organizational financial problems force reductions in the project budget.	Low	Catastrophic
It is impossible to recruit staff with the skills required for the project.	High	Catastrophic
Key staff are ill at critical times in the project.	Moderate	Serious
Faults in reusable software components have to be repaired before these components are reused.	Moderate	Serious
Changes to requirements that require major design rework are proposed.	Moderate	Serious
The organization is restructured so that different management are responsible for the project.	High	Serious
The database used in the system cannot process as many transactions per second as expected.	Moderate	Serious
The time required to develop the software is underestimated.	High	Serious
Software tools cannot be integrated.	High	Tolerable
Customers fail to understand the impact of requirements changes.	Moderate	Tolerable
Required training for staff is not available.	Moderate	Tolerable
The rate of defect repair is underestimated.	Moderate	Tolerable
The size of the software is underestimated.	High	Tolerable
Code generated by code generation tools is inefficient.	Moderate	Insignificant

6. Risk Planning Risk planning involves developing specific strategies for managing each identified risk. These strategies fall into three categories:

- **Avoidance Strategies:** Aim to reduce the probability of the risk occurring.
- **Minimization Strategies:** Focus on reducing the impact of the risk on the project or product if it does occur.
- **Contingency Plans:** These are pre-defined plans to deal with a risk if it materializes.

Managers often consider "what-if" questions during risk planning to anticipate potential scenarios, such as:

- What if several engineers are simultaneously ill?
- What if an economic downturn leads to a 20% budget cut for the project?

- What if the performance of open-source software proves inadequate, and the sole expert on that software leaves?
- What if the company supplying and maintaining software components ceases operations?
- What if the customer fails to deliver revised requirements as predicted?

Strategies to Help Manage Risk:

Risk	Strategy
Organizational financial problems	Prepare a briefing document for senior management, demonstrating the project's significant contribution to business goals and presenting reasons why budget cuts would not be cost-effective.
Recruitment problems	Alert the customer to potential difficulties and possible delays; investigate the option of buying-in components.
Staff illness	Reorganize the team to create more overlap in work, enabling team members to understand each other's jobs better.
Defective components	Replace potentially defective components with bought-in components that have known reliability.
Requirements changes	Derive traceability information to assess the impact of requirements changes; maximize information hiding within the system design.
Organizational restructuring	Prepare a briefing document for senior management, illustrating how the project significantly contributes to the business's goals.
Database performance	Investigate the possibility of purchasing a higher-performance database.
Underestimated development time	Investigate buying-in components or utilizing a program generator.

7. Risk Monitoring Risk monitoring involves regularly assessing each identified risk to determine if its probability has increased or decreased. It also includes evaluating whether the potential effects of the risk have changed. Key risks should be a standing agenda item for discussion during management progress meetings.

Risk Indicators:

Risk Type	Potential Indicators
Estimation	Failure to meet agreed schedules; failure to clear reported defects.
Organizational	Organizational gossip; lack of action by senior management.
People	Poor staff morale; poor relationships among team members; high staff turnover.
Requirements	Many requirements change requests; customer complaints.
Technology	Late delivery of hardware or support software; many reported technology problems.
Tools	Reluctance by team members to use tools; complaints about CASE tools; demands for higher-powered workstations.

Managing People

1. Importance of People Management People are considered the most valuable assets of an organization. The core tasks of a manager are fundamentally people-oriented, meaning that effective management is impossible without a sufficient understanding of human dynamics. Poor people management is a significant contributing factor to project failures.

2. People Management Factors Successful people management in a project context is guided by several key factors:

- **Consistency:** All team members should be treated comparably, avoiding favoritism or discrimination.
- **Respect:** Managers should acknowledge and respect the diverse skills and differences among team members.
- **Inclusion:** All team members should be involved in discussions, and their views must be considered.
- **Honesty:** Managers should consistently be honest about both the successes and challenges within a project.

3. Motivating People A critical role of a manager is to motivate individuals working on a project. Motivation involves structuring the work and the working environment in a way that encourages people to perform effectively. Unmotivated individuals tend to work slowly, are more prone to making mistakes, and do not contribute positively to the broader goals of the team or organization.

Motivation is a complex concept, but it is often understood in terms of different types of needs:

- **Basic Needs:** Such as physiological requirements (food, sleep) and safety.
- **Personal Needs:** Including respect and self-esteem.
- **Social Needs:** The desire to be accepted as part of a group.

4. Human Needs Hierarchy and Satisfaction in Software Development In software development groups, basic physiological and safety needs are generally not a concern. Managers can focus on satisfying higher-level needs:

- **Social Needs:** These can be met by providing communal facilities and enabling informal communication, for instance, through social networking.
- **Esteem Needs:** Satisfied through recognizing achievements and providing appropriate rewards.
- **Self-Realization Needs:** Addressed by offering training opportunities (as people desire to learn more) and assigning responsibility.

Case Study: Individual Motivation (Alice and Dorothy) Alice, a software project manager, leads a team developing assistive technology products. The project initially progresses well, with strong team relationships and creative ideas, such as a peer-to-peer messaging system. However, after several months, Alice observes a decline in Dorothy's performance (late arrivals, deteriorating work quality, reduced communication). Dorothy, a hardware design expert, later admits to losing interest in her role. She had anticipated developing and utilizing her hardware interfacing skills but found herself primarily working as a C programmer due to the chosen product direction. Despite finding the work challenging, Dorothy is concerned about not advancing her interfacing skills, fearing difficulty in finding future jobs requiring

them. Her reduced communication stems from not wanting to upset the team by revealing her thoughts about future career prospects.

Comments on the Case Study

- **Addressing Unacceptable Work:** Failing to address unacceptable work performance can lead to dissatisfaction among other group members, who may feel they are unfairly carrying the burden.
- **Personal Difficulties:** Personal challenges can significantly impact motivation by making it difficult for individuals to concentrate on their work. While support and time are needed to resolve these issues, it is important to clarify that employees still bear responsibility to their employer.
- **Alice's Resolution:** Alice responds by granting Dorothy more design autonomy and arranging software engineering training courses, providing her with more opportunities relevant to her career development.

5. Personality Types and Motivation While the needs hierarchy provides a framework, motivation in practice is more nuanced and should also account for different personality types:

- **Task-Oriented People:** Motivated primarily by the work itself. In software engineering, the work is the inherent motivation.
- **Interaction-Oriented People:** Motivated by the presence and actions of their co-workers. They enjoy going to work because of the people they work with.
- **Self-Oriented People:** Primarily motivated by personal success and recognition. The work serves as a means to achieve individual goals, such as financial gain, leisure pursuits, or travel.

6. Motivation Balance Individual motivation is typically a blend of elements from all personality types, and this balance can shift depending on personal circumstances and external events. Beyond personal factors, people are also motivated by being part of a group and its culture.

Teamwork

1. Importance of Teamwork in Software Engineering Most software engineering activities are inherently group-based. The demanding development schedules for most non-trivial software projects mean they cannot be completed by a single individual working alone. A successful group is characterized by cohesion and a strong team spirit, where members are motivated by both the group's success and their personal goals. Group interaction is a crucial determinant of overall group performance. It's important to note that flexibility in group composition is often limited, requiring managers to make the best use of available personnel.

2. Group Cohesiveness In a cohesive group, members prioritize the group's importance over any individual within it. The advantages of group cohesiveness include:

- **Shared Quality Standards:** The group members can collaboratively develop and enforce their own quality standards.

- **Mutual Learning and Understanding:** Team members learn from each other and become familiar with each other's work, reducing inhibitions caused by lack of knowledge.
- **Knowledge Sharing and Continuity:** Knowledge is shared across the team, ensuring continuity even if a group member departs.
- **Encouragement of Improvement:** Refactoring and continuous improvement are encouraged.
- **Collective Problem Solving:** Group members work together to deliver high-quality results and resolve issues, irrespective of which individual initially created a design or program.

Team Spirit Example (Alice's Actions) Alice, an experienced project manager, actively fosters group cohesiveness and team spirit. For a new product development, she involves all team members in product specification and design by encouraging them to discuss potential technologies with elderly family members and inviting these family members to meet the development group. She also organizes monthly informal lunches for the team to discuss concerns and build relationships. During these lunches, Alice shares organizational news, and team members summarize their work, discussing general topics. Additionally, every few months, Alice arranges a two-day 'away day' for 'technology updating' at an off-site hotel, where team members present on relevant technologies, allowing ample time for discussion and social interaction.

3. The Effectiveness of a Team The effectiveness of a team is influenced by several key factors:

- **The People in the Group:** A project group needs a diverse mix of individuals with different skills, as software development encompasses various activities such as client negotiation, programming, testing, and documentation.
- **The Group Organization:** The group should be structured in a way that allows individuals to contribute optimally and ensures tasks are completed as expected.
- **Technical and Managerial Communications:** Excellent communication is vital, both among group members and between the software engineering team and external project stakeholders.

4. Selecting Group Members A manager or team leader's primary responsibility is to build a cohesive group and organize it for effective collaboration. This involves creating a team with the right balance of technical skills and personalities, and then structuring the group to facilitate effective teamwork.

Assembling a Team: Constraints It is not always possible to appoint ideal candidates to a project due to various constraints:

- **Budget Limitations:** Project budgets may not permit the hiring of highly paid staff.
- **Availability of Experienced Staff:** Individuals with the precise required experience may not be available.
- **Employee Skill Development:** Organizations may choose to develop existing employee skills through project involvement. Managers must operate within these constraints, particularly when there is a shortage of trained staff.

Group Composition and Personality Types A group composed solely of members with the same primary motivation type can face problems:

- **Task-Oriented Only:** Everyone might want to pursue their own individual tasks without sufficient coordination.
- **Self-Oriented Only:** Everyone might aspire to be the leader, leading to power struggles.
- **Interaction-Oriented Only:** Excessive socializing might occur, with insufficient focus on actual work. An effective group, therefore, requires a balance of all personality types. This can be challenging to achieve, as software engineers are often predominantly task-oriented. Interaction-oriented individuals are particularly valuable because they can detect and de-escalate tensions within the team.

Group Composition Example (Alice's Team) Alice, recognizing the importance of complementary personalities, assessed potential group members during interviews to determine if they were task-oriented, self-oriented, or interaction-oriented. Believing herself to be primarily self-oriented (viewing the project as a path to promotion), she aimed to recruit one or two interaction-oriented individuals to balance the team, with task-oriented members filling the remaining roles. Her final team composition was:

- Alice — self-oriented
- Brian — task-oriented
- Bob — task-oriented
- Carol — interaction-oriented
- Dorothy — self-oriented
- Ed — interaction-oriented
- Fred — task-oriented

5. Group Organization The way a group is organized significantly impacts its decision-making processes, information exchange methods, and interactions with external project stakeholders. Key questions regarding group organization include:

- Should the project manager also serve as the technical leader of the group?
- Who will participate in critical technical decisions, and how will these decisions be made?
- How will interactions with external stakeholders and senior company management be managed?
- How can groups integrate members who are not physically co-located?
- How can knowledge be effectively shared across the group?

While small software engineering groups often adopt informal structures, large projects may necessitate a hierarchical organization with different groups responsible for distinct sub-projects. Agile development, however, typically relies on informal groups, based on the principle that rigid formal structures can inhibit information exchange.

Informal Groups In an informal group structure:

- The group operates as a collective, reaching consensus on system-affecting decisions.
- The group leader primarily serves as the external interface and does not typically allocate specific work items.

- Work is discussed by the entire group, and tasks are assigned based on individual ability and experience. This approach is particularly successful for groups where all members are experienced and competent.

6. Group Communications Effective communication is vital for successful group collaboration. It is essential for exchanging information on work status, design decisions, and changes to previous decisions. Good communication also strengthens group cohesion by fostering understanding among members.

Several factors influence the quality and effectiveness of group communications:

- **Group Size:** Larger groups generally find it more challenging for members to communicate effectively with each other.
- **Group Structure:** Communication tends to be better in informally structured groups compared to hierarchically organized ones.
- **Group Composition:** Communication improves when groups have a mix of different personality types and when they are mixed-gender rather than single-sex.
- **Physical Work Environment:** A well-organized workplace can significantly encourage communication.

UNIT-IV: Project Management: Risk Management, Managing People, Teamwork

Chapter 23 – Project Planning

1. Project Planning

Project planning is a fundamental aspect of software engineering that involves several key activities.

- **Definition:** Project planning encompasses the process of breaking down the overall work into smaller, manageable parts and assigning these parts to specific members of the project team. It also includes anticipating potential problems that may arise during the project and preparing tentative solutions to address these issues proactively.
- **Purpose of the Project Plan:** The project plan, which is developed at the beginning of a project, serves multiple critical functions. It is used to communicate to both the project team and customers how the work will be executed. Furthermore, it acts as a tool to help assess the ongoing progress of the project.
- **Planning Stages:** Project planning occurs at different stages throughout the lifecycle of a software project:
 - **Proposal Stage (Proposal Planning):**
 - **Timing:** This stage occurs when an organization is bidding for a contract to develop or supply a software system.
 - **Information Basis:** Planning may be necessary even with only outline software requirements available.
 - **Aim:** The primary objective of planning at this stage is to generate information that will be used to set a price for the system for the customers.

- **Pricing Consideration:** Project pricing involves estimating the development cost of the software, taking into account various factors such as staff costs, hardware costs, and software costs.
- **Project Startup Phase (Project Startup Planning):**
 - **Timing:** This phase begins once a contract has been secured.
 - **Information Basis:** At this stage, there is typically more detailed knowledge about the system requirements, although design or implementation information may not yet be available.
 - **Aim:** The goal is to create a plan with sufficient detail to make informed decisions regarding the project budget and staffing. This plan forms the basis for allocating project resources.
 - **Monitoring:** The startup plan should also define mechanisms for project monitoring.
 - **Applicability:** Even for agile development methodologies, a startup plan is necessary to facilitate resource allocation to the project.
- **Development Planning (Periodic Modification):**
 - **Timing:** Planning is an ongoing, iterative process that continues periodically throughout the project.
 - **Adaptation:** The project plan should be regularly revised and amended as the project progresses, incorporating new information gained about the software and its development.
 - **Revisions:** This regular amendment includes revising the project schedule, cost estimates, and risk assessments. Changes in business goals can also necessitate re-planning of projects.
- **Project Plans (in Plan-Driven Development):** In a plan-driven development project, a comprehensive project plan is created that outlines the available resources, the work breakdown, and a schedule for completing the work.
 - **Plan Sections:** A typical project plan includes the following sections:
 - Introduction
 - Project Organization
 - Risk Analysis
 - Hardware and Software Resource Requirements
 - Work Breakdown
 - Project Schedule
 - Monitoring and Reporting Mechanisms
 - **Project Plan Supplements:** Several supplementary plans may accompany the main project plan to address specific areas
refer source
- **The Planning Process:** Project planning is inherently an iterative process that commences with the creation of an initial project plan during the startup phase.
 - **Inevitability of Change:** Plan changes are considered inevitable. As more information becomes available about the system and the project team, the plan should be regularly revised to reflect changes in requirements, schedule, and risks. Business goal shifts can also lead to widespread project re-planning.
- **Planning Assumptions:** When defining a project plan, it is crucial to make realistic rather than overly optimistic assumptions.
 - **Problems:** Problems are almost always encountered during a project, inevitably leading to delays.

- **Contingency:** Initial assumptions and scheduling should therefore account for unexpected problems. Contingency should be built into the plan to prevent serious disruption to the delivery schedule if unforeseen issues arise.
- **Risk Mitigation:** If serious problems occur during development that are likely to cause significant delays, risk mitigation actions must be initiated to reduce the chances of project failure.
 - **Re-planning:** In conjunction with these mitigation actions, the project must be re-planned. This may involve renegotiating project constraints and deliverables with the customer. A new schedule for work completion must also be established and agreed upon with the customer.

2. Software Pricing

Software pricing involves determining the cost a customer will be charged for a software system.

- **Definition/Estimation Purpose:** Estimates are primarily made to ascertain the cost, from the developer's perspective, of producing a software system. This includes considering hardware, software, travel, training, and effort costs.
- **Relationship between Development Cost and Price:** There is not a straightforward, simple relationship between the actual development cost and the final price charged to the customer. Broader organizational, economic, political, and business considerations significantly influence the ultimate price.
- **Factors Affecting Software Pricing:** Various factors beyond direct development costs can influence software pricing:

Factor	Description
Contractual terms	If a customer permits the developer to retain ownership of the source code and allows its reuse in other projects, the price charged may be lower than if the source code is entirely handed over to the customer.
Cost estimate uncertainty	Should an organization be uncertain about the accuracy of its cost estimate, it may increase its price by adding a contingency amount above its usual profit margin.
Financial health	Developers facing financial difficulties might lower their price to secure a contract. In such situations, achieving a smaller-than-normal profit or breaking even is preferable to going out of business, as cash flow becomes more critical than profit during difficult economic periods.
Market opportunity	A development organization might quote a low price to enter a new segment of the software market. Accepting a low profit on a current project could create opportunities for greater profit in the future, and the experience gained may also aid in developing new products.
Requirements volatility	If the system requirements are prone to change, an organization might initially lower its price to win the contract. After the contract is awarded, higher prices can then be charged for subsequent changes to the requirements.

- **Pricing Strategies:** Different strategies can be employed for pricing software:
 - **Underpricing:**
 - A company might underprice a system to secure a contract that enables them to retain staff for future opportunities.

- Underpricing can also be used to gain access to a new market area.
- **Increased Pricing:** The price may be increased when a buyer requests a fixed-price contract, allowing the seller to account for unexpected risks by building in a higher margin.
- **Pricing to Win:** This strategy involves pricing the software based on what the software developer believes the buyer is willing to pay.
 - If the perceived willingness to pay is less than the estimated development costs, the software's functionality may be reduced accordingly, with the expectation that extra functionality can be added in a later release at additional cost.
 - Additional costs for requirements changes may also be priced at a higher level to compensate for any shortfall in the original price.

3. Plan-Driven Development

Plan-driven development is a traditional and structured approach to software engineering.

- **Definition:** Plan-driven, or plan-based, development is an approach to software engineering where the entire development process is meticulously planned in detail. It is founded on engineering project management techniques and represents the 'traditional' method for managing large-scale software development projects. A project plan is created that meticulously records the work to be performed, identifies who will execute it, specifies the development schedule, and details the work products. Managers leverage this plan to support project decision-making and as a mechanism for measuring progress.
- **Pros and Cons:**
 - **Arguments in favor (Pros):** The main advantages of a plan-driven approach are that early planning allows for thorough consideration of organizational factors, such as staff availability and other concurrent projects. Furthermore, potential problems and interdependencies are identified and addressed before the project commences, rather than emerging once development is underway.
 - **Principal Argument against (Cons):** The primary drawback of plan-driven development is that many early decisions often need to be revised due to changes in the environment in which the software is being developed and will be used.

4. Project Scheduling

Project scheduling is the process of organizing and temporalizing project work.

- **Definition:** Project scheduling is the process of deciding how the work within a project will be organized into distinct, separate tasks, and then determining when and how these tasks will be executed. This involves estimating the calendar time required for each task, the effort needed, and assigning individuals to work on the identified tasks. Additionally, it requires estimating the resources necessary for each task, such as server disk space, time on specialized hardware (e.g., a simulator), and the travel budget.
- **Project Scheduling Activities:** Key activities involved in project scheduling include:
 - Splitting the project into smaller tasks and estimating the time and resources required for each task.

- Organizing tasks to run concurrently to optimize workforce utilization.
- Minimizing task dependencies to prevent delays caused by one task awaiting the completion of another.
- The process heavily relies on the project manager's intuition and experience.
- **Scheduling Problems:** Several inherent difficulties can complicate project scheduling:
 - Estimating the complexity of problems and, consequently, the cost of developing solutions is inherently challenging.
 - Productivity is not directly proportional to the number of people working on a task.
 - Adding more people to a project that is already behind schedule tends to delay it further due to increased communication overheads.
 - Unexpected events inevitably occur. Therefore, it is always crucial to include contingency in planning.
- **Schedule Presentation:** Graphical notations are commonly used to visually represent the project schedule.
 - **Task Size:** Tasks should not be excessively small; ideally, they should take approximately one to two weeks to complete.
 - **Bar Charts:** These are the most widely used representation for project schedules. They depict the schedule by showing activities or resources against a timeline.
 - **Activity Networks:** These graphical representations are used to illustrate task dependencies.
- **Project Activities (Tasks):** Project activities, also referred to as tasks, are the fundamental planning elements. Each activity possesses specific attributes:
 - A defined duration in calendar days or months.
 - An effort estimate, indicating the number of person-days or person-months required to complete the work.
 - A deadline by which the activity should be completed.
 - A clearly defined end-point, which could be the creation of a document, the convening of a review meeting, or the successful execution of all tests, among others.
- **Milestones and Deliverables:** These are important concepts for tracking project progress and output:
 - **Milestones:** Milestones are specific points within the project schedule against which progress can be assessed. An example is the handover of the system for testing.
 - **Deliverables:** Deliverables are concrete work products that are provided to the customer, such as a requirements document for the system.
- **Tasks, Durations, and Dependencies Example:** An example table illustrating tasks, their estimated effort, duration, and dependencies is provided:

Task	Effort (person-days)	Duration (days)	Dependencies
T1	15	10	-
T2	8	15	-
T3	20	15	T1 (M1)
T4	5	10	-
T5	5	10	T2, T4 (M3)
T6	10	5	T1, T2 (M4)
T7	25	20	T1 (M1)

T8	75	25	T4 (M2)
T9	10	15	T3, T6 (M5)
T10	20	15	T7, T8 (M6)
T11	10	10	T9 (M7)

5. Agile Planning

Agile planning is an iterative approach to software development.

- **Definition:** Agile methods of software development are characterized by iterative approaches where software is developed and delivered to customers in small, incremental releases.
- **Key Characteristics:** Unlike plan-driven approaches, the specific functionality of these increments is not determined in advance; instead, it is decided during the development process itself. The decision on what to include in an increment is highly dependent on the current progress and the customer's evolving priorities. Given that customer priorities and requirements frequently change, agile planning adopts a flexible approach that can accommodate these modifications.
- **Agile Planning Stages:** Agile planning typically involves two main stages:
 - **Release Planning:** This stage involves looking several months ahead to decide which features should be included in an upcoming release of the system.
 - **Iteration Planning:** This stage has a shorter-term outlook, focusing on planning the next increment of the system. This typically represents 2-4 weeks of work for the development team.
- **Approaches to Agile Planning:**
 - **Planning in Scrum:** This approach, discussed in Chapter 3 of the source material, is based on managing a project backlog (a list of items to be done) with daily reviews of progress and identified problems.
 - **The Planning Game:** Originally developed as part of Extreme Programming (XP), the planning game relies on "user stories" as a measure of progress within the project.
 - **Story-based Planning:** The planning game is centered around user stories, which articulate the features that should be incorporated into the system.
 - **Effort Points:** The project team reads and discusses these stories, then ranks them based on their estimated implementation time. Stories are assigned "effort points" to reflect their size and difficulty.
 - **Velocity:** The number of effort points implemented per day is measured, providing an estimate of the team's "velocity." This velocity then allows for the estimation of the total effort required to implement the entire system.
- **Release and Iteration Planning (Detailed):**
 - **Release Planning:** This involves selecting and refining the user stories that will define the features for a system release, and determining the order in which these stories should be implemented.
 - **Iteration Planning:** Stories designated for implementation in each iteration are chosen. The number of stories selected is guided by the team's velocity to ensure they can be delivered within the typical iteration timeframe (usually 2 or 3 weeks).

- **Task Allocation:** During the task planning stage, developers break down user stories into more granular development tasks.
 - **Task Size:** A development task should ideally take between 4 and 16 hours to complete.
 - **Listing Tasks:** All tasks necessary to implement the stories within a given iteration are listed.
 - **Developer Sign-up:** Individual developers then sign up for the specific tasks they intend to implement.
 - **Benefits:** This approach offers several advantages:
 - The entire team gains a comprehensive overview of all tasks to be completed within an iteration.
 - Developers develop a sense of ownership over their chosen tasks, which can serve as a strong motivator for task completion.
- **Software Delivery:** A software increment is consistently delivered at the end of each project iteration.
 - **Scope Adjustment:** If the features intended for an increment cannot be fully completed within the allotted time, the scope of work is reduced accordingly.
 - **Fixed Schedule:** Critically, the delivery schedule is never extended.
- **Agile Planning Difficulties:** Despite its benefits, agile planning faces certain challenges:
 - **Customer Involvement:** It heavily relies on continuous customer involvement and availability, which can be difficult to manage, as customer representatives may have other priorities.
 - **Customer Familiarity:** Some customers may be more accustomed to traditional project plans and may find it challenging to engage effectively in an agile planning process.
- **Agile Planning Applicability:** Agile planning is most effective under specific conditions:
 - It functions well with small, stable development teams who can regularly convene to discuss the stories to be implemented.
 - However, its effectiveness diminishes when teams are large, geographically distributed, or when team membership changes frequently, as collaborative planning, essential for agile project management, becomes practically impossible for everyone to participate.

6. Estimation Techniques

Organizations require robust methods to estimate software effort and cost accurately.

- **Purpose:** The primary purpose of estimation techniques is to determine the effort and cost required for software development.
- **Types of Techniques:** There are two main categories of techniques used for estimation:
 - **Experience-based Techniques:**
 - **Approach:** In this method, the estimation of future effort requirements is based on the manager's experience from past projects and their knowledge of the application domain. Essentially, the manager makes an informed judgment regarding the likely effort needed.
 - **Process:** Typically, the deliverables to be produced and the various software components or systems to be developed are identified. These

are then documented in a spreadsheet, estimated individually, and aggregated to compute the total required effort.

- **Group Estimation:** It is often beneficial to involve a group of people in the effort estimation process and ask each member to explain their rationale behind their estimate.
- **Problems with Experience-based Approaches:** The main challenge with experience-based techniques is that a new software project may bear little resemblance to previous projects. Software development evolves rapidly, and projects frequently incorporate unfamiliar techniques such as web services, application system configuration, or HTML5. If one lacks prior experience with these specific techniques, previous project experience may not adequately inform the effort required, making it difficult to produce accurate cost and schedule estimates.

- **Algorithmic Cost Modelling:**

- **Approach:** This method employs a formulaic approach to calculate the project effort. The computation is based on estimates of product attributes (such as size) and process characteristics (such as the experience level of the involved staff).
- **Formula:** Cost is estimated as a mathematical function of product, project, and process attributes, whose values are estimated by project managers. A common representation is: $\text{Effort} = A \times \text{Size}^B \times M$
Where:
 - A is an organization-dependent constant.
 - B reflects the disproportionate effort required for larger projects.
 - M is a multiplier that accounts for various product, process, and people attributes.
- **Product Attribute:** The most commonly utilized product attribute for cost estimation in these models is code size.
- **Model Variation:** Most algorithmic models are structurally similar but employ different values for the constants A, B, and the multiplier M.
- **Estimation Accuracy:**
 - The precise size of a software system can only be known accurately upon its completion.
 - Several factors influence the final system size, including the use of reused systems and components, the programming language chosen, and the distribution of the system.
 - As the development process progresses, the size estimate generally becomes more accurate.
 - It is important to note that the estimates for the factors contributing to B and M are subjective and depend on the judgment of the estimator.
- **Effectiveness of Algorithmic Models:**
 - Algorithmic cost models provide a systematic framework for estimating the effort needed to develop a system.
 - However, these models are often complex and challenging to use, involving numerous attributes with considerable scope for uncertainty in estimating their values.

- This complexity has limited the practical application of algorithmic cost modeling to a relatively small number of large corporations, predominantly those involved in defense and aerospace systems engineering.

7. COCOMO Cost Modelling

COCOMO (Constructive Cost Model) is a widely recognized empirical model for software cost estimation.

- **Overview:**
 - **Empirical Basis:** COCOMO is an empirical model rooted in extensive project experience.
 - **Independence:** It is a well-documented and "independent" model, meaning it is not tied to any specific software vendor.
 - **History:** COCOMO has a long history, originating from its initial version published in 1981 (COCOMO-81), and evolving through various instantiations to COCOMO 2.
 - **COCOMO 2:** This version accounts for diverse approaches to software development, including reuse.
- **COCOMO 2 Models (Sub-models):** COCOMO 2 integrates a range of sub-models that generate progressively more detailed software estimates as the project progresses. These sub-models are:
 - **Application Composition Model:** Used when software is assembled from existing parts, typically supporting prototyping projects and those with extensive reuse.
 - **Basis:** Based on standard estimates of developer productivity in application (object) points per month.
 - **Tools:** It also considers the use of software tools.
 - **Formula:** The effort in person-months (PM) is calculated as: $PM = (NAP \times (1 - \%reuse/100)) / PROD$ Where:
 - PM is the effort in person-months.
 - NAP is the number of application points.
 - PROD is the productivity.
 - **Application-point Productivity (PROD) Table:** Productivity (NAP/month) varies based on developer experience, capability, ICASE maturity, and capability:

Developer's experience and capability	ICASE maturity and capability	PROD (NAP/month)
Very low	Very low	4
Low	Low	7
Nominal	Nominal	13
High	High	25
Very high	Very high	50

* ****Early Design Model**:** Estimates can be made after requirements have been agreed upon, but before design has started.

* ****Basis**:** This model uses a standard formula characteristic of algorithmic models:

$$PM = A \times Size^B \times M$$

Where:

- * `A` is typically `2.94` in initial calibration.
- * `Size` is measured in KLOC (Thousands of Lines of Code).
- * `B` varies from `1.1` to `1.24`, depending on project novelty, development flexibility, risk management approaches, and process maturity.
- * `M` is a multiplier calculated as:

$$M = PERS \times RCPX \times RUSE \times PDIF \times PREX \times FCIL \times SCED`$$
- * ****Multipliers (M)****: These multipliers reflect various project attributes:
 - * `RCPX`: Product reliability and complexity.
 - * `RUSE`: The required level of reuse.
 - * `PDIF`: Platform difficulty.
 - * `PREX`: Personnel experience.
 - * `PERS`: Personnel capability.
 - * `SCED`: Required schedule acceleration/compression.
 - * `FCIL`: The team support facilities.
- * ****Reuse Model****: This model considers both "black-box" code (reused without change) and "white-box" code (adapted and integrated with new code).
 - * ****Versions****: There are two main versions:
 - * ****Black-box reuse****: Where code is not modified. An effort estimate (PM) is computed.
 - * ****White-box reuse****: Where code is modified. A size estimate equivalent to the number of lines of new source code is computed, which then adjusts the size estimate for new code.
 - * ****Reuse Model Estimates 1 (for Generated Code)****: For automatically generated code, the effort (PM) is calculated as:

$$PM = (ASLOC \times AT/100) / ATPROD`$$
 Where:
 - * `ASLOC` is the number of lines of generated code.
 - * `AT` is the percentage of code automatically generated.
 - * `ATPROD` is the productivity of engineers in integrating this code.
 - * ****Reuse Model Estimates 2 (for Code Understanding and Integration)****: When code needs to be understood and integrated (modified), the equivalent size (ESLOC) is calculated as:

$$ESLOC = ASLOC \times (1 - AT/100) \times AAM`$$
 Where:
 - * `ASLOC` and `AT` are as defined previously.
 - * `AAM` is the adaptation adjustment multiplier, computed from the costs associated with changing the reused code, understanding how to integrate it, and the costs of reuse decision-making.
- * ****Post-architecture Model****: This model is used once the system architecture has been designed and more detailed information about the system becomes available.
 - * ****Formula****: It uses the same fundamental formula as the early design model, but incorporates 17 associated multipliers instead of 7, reflecting greater detail.
 - * ****Code Size Estimation****: The code size for this model is estimated by combining:
 - * The number of lines of new code to be developed.
 - * An estimate of the equivalent number of lines of new code computed using the reuse model.
 - * An estimate of the number of lines of code that require modification due to requirements changes.
 - * ****The Exponent Term (B)****: This exponent depends on five scale factors. Their sum divided by 100 is added to 1.01. An example illustrates its calculation: for a project in a new domain, with undefined process, no risk analysis, new team, and CMM level 2 rating, the scale factor would be $(4 + 1 + 5 + 3 + 3) / 100 + 1.01 = 1.17`$.
 - * ****Scale Factors used in Exponent Computation (Post-Architecture Model)****: These factors influence the exponent `B`:

Scale factor	Explanation
Architecture/risk resolution	Reflects the extent of risk analysis conducted. "Very low" indicates minimal analysis, while "extra-high" signifies a complete and thorough risk analysis.
Development flexibility	Reflects the degree of flexibility permitted in the development process. "Very low" means a prescribed process is strictly followed, whereas "extra-high" implies the client sets only general goals.
Precedentedness	Reflects the organization's prior experience with the specific type of project. "Very low" indicates no previous experience, while "extra-high" signifies complete familiarity with the application domain.
Process maturity	Reflects the organization's process maturity. Its value is often estimated by subtracting the CMM (Capability Maturity Model) process maturity level from 5.
Team cohesion	Reflects how well the development team members know and collaborate with each other. "Very low" suggests very difficult interactions, while "extra-high" indicates an integrated and highly effective team with strong synergy.

* ****Multipliers (Cost Drivers)****: In the post-architecture model, the 17 multipliers are categorized into:

* ****Product attributes****: Concerned with the required characteristics of the software product being developed.

* ****Computer attributes****: Relate to constraints imposed on the software by the hardware platform.

* ****Personnel attributes****: Account for the experience and capabilities of the individuals working on the project.

* ****Project attributes****: Address specific characteristics of the software development project itself.

* ****The Effect of Cost Drivers on Effort Estimates (Examples)****: Cost drivers significantly impact effort estimates.

* ****Example 1 (High Cost Drivers)****:

* Exponent value: 1.17

* System size (including factors for reuse and requirements volatility): 128,000 DSI (Developed Source Instructions)

* Initial COCOMO estimate without cost drivers: 730 person-months

* With high impact cost drivers:

* Reliability: Very high, multiplier = 1.39

* Complexity: Very high, multiplier = 1.3

* Memory constraint: High, multiplier = 1.21

* Tool use: Low, multiplier = 1.12

* Schedule: Accelerated, multiplier = 1.29

* Adjusted COCOMO estimate: ****2,306 person-months****

* ****Example 2 (Low Cost Drivers)****:

* Exponent value: 1.17

* Reliability: Very low, multiplier = 0.75

* Complexity: Very low, multiplier = 0.75

* Memory constraint: None, multiplier = 1

* Tool use: Very high, multiplier = 0.72

* Schedule: Normal, multiplier = 1

* Adjusted COCOMO estimate: ****295 person-months****

- **Project Duration and Staffing**: Beyond effort estimation, managers must estimate the calendar time required to complete a project and anticipate when staff will be needed.

- **Calendar Time Estimation**: Calendar time can be estimated using a COCOMO 2 formula: $TDEV = 3 \times (PM)^{(0.33 + 0.2 \times (B-1.01))}$ Where:

- T_{DEV} is the calendar time.
- PM is the computed effort.
- B is the exponent calculated as discussed previously (for the early prototyping model, B is 1). This computation predicts the nominal schedule for the project. Notably, the time required is independent of the number of people working on the project.
- **Staffing Requirements:** The number of staff required cannot be simply calculated by dividing the total development time by the required schedule. The number of people working on a project typically varies depending on the project phase. Generally, the more people involved in a project, the greater the total effort required. A very rapid increase in staffing often correlates with schedule slippage.